

Processes vs Threads and Concurrent Execution

A comprehensive guide to understanding processes, threads, multithreading, concurrency, parallelism, and how operating systems schedule execution.

Table of Contents

1. [Process vs. Thread](#)
 2. [Multithreading](#)
 3. [Context Switching](#)
 4. [Concurrency vs. Parallelism](#)
 5. [CPU Cores Explained](#)
 6. [How the OS Schedules Threads](#)
 7. [When Multithreading Helps \(and When It Doesn't\)](#)
-

Process vs. Thread

What is a Process?

A **process** is a program in execution. When you run a program (stored on your hard disk), it gets loaded into memory and becomes a process.

Key characteristics of a process:

- Has its own **memory address space** (isolated from other processes)
- Contains program code, data, and resources (file handles, network connections, etc.)
- Managed and scheduled by the operating system
- Heavyweight - takes more resources to create and manage

Example: When you open Microsoft Word, Chrome, or Spotify, each becomes a separate process.

What is a Thread?

A **thread** is a unit of execution within a process. It's the actual entity that executes program instructions.

Key characteristics of a thread:

- Shares the parent process's memory address space
- Has its own stack for local variables and function calls
- Shares the heap, global variables, and code with other threads in the same process
- Lightweight - much faster to create than a process
- **Every process must have at least one thread** (the main thread)

Analogy: If a process is a restaurant kitchen, threads are the individual chefs working in that kitchen, sharing the same workspace and ingredients.

Process vs. Thread Comparison

Aspect	Process	Thread
Memory	Own address space	Shares process memory
Resources	Own resources	Shares process resources
Creation time	Slower	Faster
Overhead	Higher	Lower
Communication	IPC (Inter-Process Communication)	Direct memory access
Isolation	Fully isolated	Not isolated (share memory)

Multithreading

What is Multithreading?

Multithreading is when a single process has multiple threads running to accomplish work more efficiently or faster.

Simple explanation: One program running multiple "workers" (threads) simultaneously to get work done.

Advantages of Multithreading

1. Improved Performance on Multi-Core Systems

On systems with multiple CPU cores, multithreading enables **parallelism** - different threads can run simultaneously on different cores, truly speeding up execution.

Example: Video encoding with 4 threads on a 4-core CPU

- Thread 1 processes frames 1-250
- Thread 2 processes frames 251-500
- Thread 3 processes frames 501-750
- Thread 4 processes frames 751-1000

All four threads work at the exact same time, completing the job ~4x faster.

2. Better Resource Utilization (Even on Single Core)

For I/O-bound tasks (tasks that wait for input/output like network, disk, or user input), multithreading keeps the CPU busy while some threads are waiting.

Example: Web browser downloading 3 files

- Thread 1 starts download → waits for network
- While Thread 1 waits, Thread 2 starts its download → waits for network
- While both wait, Thread 3 starts its download
- CPU switches between them, keeping busy instead of idling

3. Responsive User Interfaces

Separating UI from background work prevents applications from freezing.

Example: Music player

- Thread 1: Handles UI (play button, slider)
- Thread 2: Decodes and plays music in background

User can adjust volume or skip songs without the app freezing during audio processing.

Disadvantages of Multithreading

1. Synchronization Overhead

Since threads share memory, they can interfere with each other by reading/writing the same data simultaneously.

The problem: Race conditions - when two threads modify shared data, results become unpredictable.

Example:

Initial balance: \$100

Thread A reads balance: \$100

Thread A calculates: $\$100 + \$50 = \$150$

Thread B reads balance: \$100 (still sees \$100!)

Thread B calculates: $\$100 + \$30 = \$130$

Thread A writes: \$150

Thread B writes: \$130

Final balance: \$130 (should be \$180!)

Solution: Use synchronization constructs:

- **Mutexes (Mutual Exclusion):** Only one thread can access protected data at a time
- **Semaphores:** Control access to a limited number of resources
- **Locks:** Prevent concurrent access to critical sections

The overhead: These synchronization mechanisms consume CPU time and memory, and threads may wait (blocked) while another thread holds a lock.

2. Context Switching Overhead

When there are more threads than CPU cores, the OS must rapidly switch between threads. Each switch has a small cost.

What happens during context switching:

1. Save current thread's state (registers, program counter, stack pointer)
2. Load next thread's state
3. Resume execution

While modern OSes are efficient at this, excessive context switching can degrade performance.

3. Increased Complexity

Multithreaded programs are harder to:

- Write correctly (avoiding race conditions, deadlocks)
 - Debug (non-deterministic behavior)
 - Test (timing-dependent bugs)
-

Context Switching

What is Context Switching?

Context switching is when a CPU core stops working on one thread and switches to work on another thread.

Simple analogy: Imagine reading multiple books - you read Book A for 10 minutes, put it down, pick up Book B, read for 10 minutes, then back to Book A. Each time you switch books, you need a moment to remember where you were. The CPU does the same when switching threads.

When Does Context Switching Happen?

Context switches occur when:

1. **Thread's time slice expires** (timer interrupt every 10-100 milliseconds)
 2. **Thread blocks** (waiting for I/O, waiting for a lock, or calls sleep)
 3. **Thread terminates**
 4. **Higher priority thread becomes ready** (preemption)
 5. **Thread voluntarily yields** the CPU
-

Context Switching on Single-Core vs. Multi-Core

On Single-Core CPUs

Context switching is **essential** and **constant** because only one thread can execute at a time.

Example with 4 threads on 1 core:

```
Time 0-20ms: Thread A runs
Time 20-40ms: Thread B runs (context switch)
Time 40-60ms: Thread C runs (context switch)
Time 60-80ms: Thread D runs (context switch)
Time 80-100ms: Thread A runs (context switch)
... and so on
```

This is **concurrency** - threads take turns so fast it appears simultaneous, but only one actually runs at any moment.

On Multi-Core CPUs

Context switching still happens but is **reduced** compared to single-core.

Example with 4 threads on 4-core CPU:

Core 1: Thread A runs continuously

Core 2: Thread B runs continuously

Core 3: Thread C runs continuously

Core 4: Thread D runs continuously

All four threads run **truly simultaneously** (parallelism).

However, context switching still occurs when:

- You have more threads than cores (e.g., 200 threads on 4 cores)
 - Higher priority threads need CPU time
 - OS needs to schedule system processes
-

Context Switching Overhead

The overhead includes:

- Saving current thread state (registers, stack pointer)
- Loading next thread state
- Potential cache misses (new thread's data might not be in CPU cache)

Important note: The overhead itself is small (microseconds), but it adds up with frequent switching. The **real inefficiency** for CPU-intensive tasks on single core is not the switch cost but the fact that you're **not gaining any additional computational power** - you're just dividing the same CPU time.

Concurrency vs. Parallelism

These terms are often confused but describe fundamentally different concepts.

Concurrency

Definition: Managing multiple tasks at once (but not necessarily executing simultaneously).

How it works: Tasks take turns using available resources (like a single CPU core), switching so rapidly it appears simultaneous.

Real-world analogy: One chef rapidly switching between chopping vegetables, stirring soup, and checking the oven. The chef handles multiple tasks "at once" by quickly switching between them.

Technical example: 3 threads on a single CPU core

CPU Core:

[0-20ms] Thread A executes

[20-40ms] Thread B executes (context switch)

[40-60ms] Thread C executes (context switch)

[60-80ms] Thread A executes (context switch)

All three threads make progress, but only one actually runs at any instant.

Key point: Concurrency is about **dealing with** multiple things at once.

Parallelism

Definition: Actually executing multiple tasks at the exact same time.

How it works: Multiple CPU cores each execute different tasks simultaneously.

Real-world analogy: Three chefs, each doing their own task at the exact same moment - one chopping, one stirring, one plating.

Technical example: 3 threads on a 3-core CPU

Core 1: Thread A executes

Core 2: Thread B executes } All happening at the

Core 3: Thread C executes } exact same instant

Key point: Parallelism is about **doing** multiple things at once.

The Critical Difference

Aspect	Concurrency	Parallelism
Execution	Time-slicing (takes turns)	Simultaneous
Requires	Single or multi-core	Multi-core only
Goal	Manage multiple tasks efficiently	Speed up execution
Example	4 threads on 1 core	4 threads on 4 cores

Visual Summary

Concurrency (Single Core):

Time →
CPU: [A][B][C][A][B][C][A][B][C]

Threads switch rapidly but never run simultaneously.

Parallelism (Multi-Core):

Time →
Core 1: [A][A][A][A][A][A][A][A]
Core 2: [B][B][B][B][B][B][B][B]
Core 3: [C][C][C][C][C][C][C][C]

All threads run at the exact same time.

CPU Cores Explained

What is a CPU Core?

A **CPU core** is an independent processing unit that can execute instructions.

Simple definition: A core is like a worker - one worker can do one task at a time.

Multi-core CPU: Multiple independent workers on a single chip, each capable of executing instructions simultaneously.

Core Count Examples

Modern CPUs contain multiple cores:

- **Intel Core i5:** Typically 6 cores
 - **AMD Ryzen 7:** Typically 8 cores
 - **Apple M1:** 8 cores (4 performance + 4 efficiency)
 - **High-end workstations:** 16, 32, or even 64 cores
-

Single-Core vs. Multi-Core

Single-Core CPU (Older Computers)

- Only 1 execution unit
- Can only truly execute 1 instruction at a time
- Multithreading = rapid switching (concurrency only)
- No true parallelism

Multi-Core CPU (Modern Computers)

- Multiple execution units
 - Can execute multiple instructions simultaneously
 - Multithreading = actual parallel execution across cores
 - True parallelism
-

Restaurant Kitchen Analogy

Single-Core CPU = 1 chef with 1 stove

- Can only cook 1 dish at a time
- If you have 4 orders, the chef must switch between them
- Orders take longer

Quad-Core CPU = 4 chefs with 4 separate stoves

- Each chef can cook a different dish simultaneously
 - 4 orders? All 4 get cooked at the same time
 - Orders complete ~4x faster (for parallelizable tasks)
-

What Each Core Can Do

Each core can:

- Execute instructions independently
- Run one or more threads (via time-slicing)
- Perform calculations simultaneously with other cores

- Has its own L1/L2 cache (usually)

Example with 4 cores:

Core 1: Running Thread A (calculating taxes)
Core 2: Running Thread B (encoding video)
Core 3: Running Thread C (playing music)
Core 4: Running Thread D (downloading file)

All four threads execute **truly at the same time** - that's parallelism.

How to Check Your Cores

Windows: Task Manager → Performance tab

Mac: Apple Menu → About This Mac

Linux: Run `lscpu` command

Most modern computers have 4-8 cores (or more for high-end systems).

How the OS Schedules Threads

The operating system scheduler is responsible for deciding which threads run, on which cores, and for how long.

What is the OS Scheduler?

The **scheduler** is a core OS component that manages CPU allocation. Think of it as an air traffic controller deciding which planes land on which runways and when.

The scheduler decides:

1. Which thread runs next
 2. On which CPU core
 3. For how long (time slice)
-

Key Scheduling Concepts

1. Time Slices (Quantum)

Each thread gets a small chunk of CPU time called a **time slice** or **quantum** (typically 10-100 milliseconds).

Example:

Thread A runs for 20ms → switch
Thread B runs for 20ms → switch
Thread C runs for 20ms → switch
Back to Thread A...

After its time slice expires, the scheduler preempts the thread and switches to another.

2. Thread States

Threads exist in different states:

- **Running:** Currently executing on a CPU core
- **Ready:** Ready to run, waiting to be scheduled
- **Blocked/Waiting:** Waiting for something (I/O, lock, sleep, event)
- **Terminated:** Finished execution

Important: The scheduler only schedules threads in the **Ready** state. Blocked threads don't consume CPU time.

3. Run Queue

The OS maintains a **run queue** (or multiple queues) containing all threads that are ready to run.

Ready Queue: [Thread A, Thread B, Thread C, Thread D, Thread E]

The scheduler picks threads from this queue based on its algorithm.

Thread Priority

Each thread has a **priority** level (a number indicating importance):

- **High priority:** System-critical tasks, real-time processes
- **Normal priority:** Regular user applications
- **Low priority:** Background tasks, maintenance

How it works: Higher priority threads get scheduled first and receive more CPU time.

Example:

Priority 10 (high): [Thread A]

Priority 5 (medium): [Thread B, Thread C]

Priority 1 (low): [Thread D]

Scheduler picks Thread A first, then B and C, lastly D

Common Scheduling Algorithms

1. Round Robin (Simple Fairness)

Each thread gets equal time slices in rotation.

Time: 0-20ms → Thread A

Time: 20-40ms → Thread B

Time: 40-60ms → Thread C

Time: 60-80ms → Thread A (back to start)

Pros: Simple and fair

Cons: Doesn't consider priority or thread behavior

2. Priority-Based Scheduling

Higher priority threads always run before lower priority ones.

Queue:

Priority 10: [Thread A]

Priority 5: [Thread B, Thread C]

Priority 1: [Thread D]

Scheduler picks Thread A, then B/C, finally D

Pros: Important tasks get CPU first

Cons: Low-priority threads might **starve** (never run if high-priority threads keep arriving)

3. Multi-Level Feedback Queue (Modern OSes)

Combines priority with dynamic adjustment based on thread behavior.

How it works:

- Threads start at a default priority
- If a thread uses its full time slice (CPU-intensive), its priority **decreases**
- If a thread blocks quickly (I/O-bound), its priority **increases**

Why? I/O-bound threads (like UI interactions) should run quickly and return to waiting, keeping the system responsive. CPU-intensive threads can afford to wait.

Example:

Thread A (UI clicks) - blocks quickly → priority increases → feels responsive

Thread B (video render) - uses full slice → priority decreases → runs in background

4. Completely Fair Scheduler (CFS) - Linux

Tries to give each thread a fair share of CPU time based on:

- How much CPU time it has already received
- Its priority/nice value

Core idea: Threads that have received less CPU time recently get scheduled sooner. This ensures fairness over time.

Multi-Core Scheduling

With multiple cores, the scheduler also decides **which core** runs **which thread**.

CPU Affinity

The OS tries to keep threads on the same core when possible because:

- CPU caches contain that thread's data
- Switching cores means reloading cache (cache miss = slower)

Example with 4 cores:

Core 1: Thread A, Thread E (prefer keeping Thread A on Core 1)
Core 2: Thread B, Thread F
Core 3: Thread C, Thread G
Core 4: Thread D, Thread H

Load balancing: If one core is overloaded while others are idle, the scheduler moves threads to balance the load.

When Scheduling Happens

The scheduler runs and makes decisions when:

1. **Time slice expires** (timer interrupt fires every few milliseconds)
 2. **Thread blocks** (waiting for I/O, lock, or calls sleep)
 3. **Thread terminates** (finishes execution)
 4. **Higher priority thread becomes ready** (preempts current thread)
 5. **Thread voluntarily yields** the CPU
-

Real-World Scheduling Example

Scenario:

- 4-core CPU
- 10 threads running

Core 1: Running Thread A (priority 5)
Core 2: Running Thread B (priority 8)
Core 3: Running Thread C (priority 5)
Core 4: Running Thread D (priority 3)

Ready Queue: [Thread E (pri 10), Thread F (pri 7), Thread G (pri 5), ...]

Event 1: Thread A's time slice expires (20ms passed)

- Scheduler examines ready queue
- Thread E has priority 10 (higher than A's 5)
- **Context switch:** Thread E replaces Thread A on Core 1
- Thread A goes back to ready queue

Event 2: Thread B blocks (waiting for disk I/O)

- Core 2 becomes free immediately (no waiting for time slice)
- Scheduler picks Thread F (next highest priority: 7)
- Thread F runs on Core 2
- Thread B moves to blocked state (not in ready queue)

Event 3: Disk I/O completes for Thread B

- Thread B becomes ready again
 - OS might increase Thread B's priority (I/O-bound behavior)
 - Thread B enters ready queue, waiting to be scheduled
-

Different OS Schedulers

Different operating systems use different scheduling algorithms:

- **Windows:** Priority-based scheduler with 32 priority levels
 - **Linux:** Completely Fair Scheduler (CFS)
 - **macOS:** Mach scheduler (priority-based with time-sharing)
 - **Real-Time OSes:** Deterministic schedulers with guaranteed response times
-

Scheduler Summary

The OS scheduler:

1. Maintains queues of ready threads
2. Uses algorithms balancing fairness, priority, and responsiveness
3. Assigns threads to available CPU cores
4. Switches threads when time expires or threads block
5. Tries to keep threads on the same core (cache affinity)
6. Dynamically adjusts priorities based on thread behavior

All of this happens **thousands of times per second**, completely transparently to running programs.

When Multithreading Helps (and When It Doesn't)

Understanding when to use multithreading is crucial for writing efficient programs.

✅ Multithreading HELPS When:

1. I/O-Bound Tasks (Even on Single Core)

Tasks that spend time waiting for input/output operations benefit enormously from multithreading.

Examples:

- Web servers handling multiple requests
- Downloading multiple files
- Reading from disk or database
- Network communication
- Waiting for user input

Why it helps: While one thread waits (blocked), other threads can use the CPU. This keeps the CPU busy instead of idle.

Example: Web browser

Thread 1: Downloads image → waits for network
Thread 2: Downloads CSS → waits for network } CPU switches between
Thread 3: Downloads JS → waits for network } them, staying busy

Without multithreading, downloads would happen sequentially with CPU sitting idle during network waits.

2. CPU-Intensive Tasks on Multi-Core Systems

Tasks requiring heavy computation benefit from parallelism across multiple cores.

Examples:

- Video encoding
- 3D rendering
- Scientific simulations
- Image processing
- Machine learning training

- Data analysis

Why it helps: Work is divided among cores, each running simultaneously, achieving real speedup.

Example: Encoding a 1000-frame video on 4-core CPU

Core 1: Frames 1-250
Core 2: Frames 251-500 } All encoding
Core 3: Frames 501-750 } at the same time
Core 4: Frames 751-1000 }

Result: ~4x faster than single-threaded encoding.

3. Responsive User Interfaces

Separating UI from background work prevents freezing.

Examples:

- Music players (UI + audio processing)
- IDEs (UI + code compilation)
- Games (rendering + physics + AI)
- Document editors (UI + spell check)

Why it helps: UI thread stays responsive to user input while background threads handle heavy processing.

Example: Photo editor

Thread 1 (UI): Handles button clicks, slider movements - always responsive
Thread 2 (Worker): Applies filters to image - takes time but doesn't freeze UI

✗ Multithreading Does NOT Help When:

1. CPU-Intensive Tasks on Single Core

Pure computation with no waiting gains nothing (and might lose) from multithreading on single-core systems.

Examples:

- Calculating prime numbers
- Complex mathematical computations
- Encryption/decryption

- Sorting large arrays
- Any pure CPU work with no I/O

Why it doesn't help: You only have ONE core. Multithreading just divides the same computational power plus adds context switching overhead.

Example: Calculate first 1 million primes on single-core

- 1 thread: Takes 10 seconds
- 4 threads: Still takes ~10 seconds (or slightly more due to overhead)

The math: 1 core $\times$ 100% utilization = same total work done regardless of thread count.

2. Tasks That Can't Be Parallelized

Some tasks have inherent dependencies where steps must happen in order.

Examples:

- Sequential algorithms (step N requires step N-1's result)
- Linked list traversal
- Recursive computations with dependencies
- Pipeline processing where each stage feeds the next

Why it doesn't help: You can't do step 2 until step 1 completes, regardless of thread count.

Example: Cooking pasta

```
Step 1: Boil water    } Must happen  
Step 2: Add pasta     } in this exact  
Step 3: Wait 10 min   } sequence  
Step 4: Drain         }
```

No amount of threads lets you add pasta before water boils.

3. When Synchronization Overhead Exceeds Benefits

If threads spend more time waiting for locks than doing useful work, multithreading hurts performance.

Example: 10 threads all trying to update the same shared counter

```
Thread 1: Wait for lock... acquire... increment... release
Thread 2: Wait for lock... acquire... increment... release
Thread 3: Wait for lock... acquire... increment... release
...
```

All threads wait in line for the lock. Result: slower than single-threaded.

The problem: Too much contention for shared resources creates a bottleneck.

4. Tasks with Tiny Workloads

If the work per thread is very small, thread creation and management overhead exceeds the benefit.

Example: Sum an array of 100 numbers using 10 threads

- Thread creation/setup time > actual computation time
- Better to just do it in one thread

Rule of thumb: Multithreading makes sense when per-thread work is substantial (milliseconds to seconds, not microseconds).

The Golden Rules

Rule 1: I/O-Bound Tasks

Waiting around for I/O? → Multithreading helps (even on 1 core)

The CPU can work on other threads while some wait for network, disk, or user input.

Rule 2: CPU-Bound Tasks

Pure computation? → Multithreading only helps with multiple cores

On single core: No benefit (just overhead)

On multi-core: Benefits scale with core count (up to task parallelism limit)

Rule 3: Parallelizability

Can the task be divided into independent sub-tasks? → Multithreading can help

If yes + multi-core = speedup

If no = no speedup regardless of threads/cores

Decision Matrix

Task Type	Single Core	Multi-Core	Verdict
I/O-bound	✓ Helps	✓ Helps	Always beneficial
CPU-bound	✗ No benefit	✓ Helps	Only with multiple cores
Sequential	✗ No benefit	✗ No benefit	Can't parallelize
High contention	✗ Hurts	✗ Hurts	Too much locking
Tiny workload	✗ Overhead	✗ Overhead	Not worth it

Summary

Key Takeaways

1. Processes vs. Threads:

- Processes are isolated programs with their own memory
- Threads are lightweight execution units sharing process memory
- Every process has at least one thread

2. Multithreading:

- Multiple threads in one process
- Pros: Better performance (multi-core), resource utilization (I/O), responsiveness
- Cons: Synchronization overhead, context switching cost, complexity

3. Context Switching:

- OS saves one thread's state and loads another's
- Happens on both single-core (constantly) and multi-core (when threads > cores)
- Small overhead per switch, but adds up with frequent switching

4. Concurrency vs. Parallelism:

- Concurrency = managing multiple tasks (time-slicing on single core)
- Parallelism = executing multiple tasks simultaneously (multi-core only)

- Concurrency is about dealing with, parallelism is about doing

5. CPU Cores:

- Independent processing units within a CPU
- More cores = more simultaneous execution
- Modern CPUs have 4-64+ cores

6. OS Scheduling:

- Scheduler decides which threads run, where, and for how long
- Uses time slices (10-100ms), priorities, and fairness algorithms
- Common algorithms: Round Robin, Priority-based, Multi-level Feedback Queue, CFS
- Happens thousands of times per second

7. When to Use Multithreading:

-  I/O-bound tasks (always beneficial)
-  CPU-bound on multi-core (scales with cores)
-  Responsive UIs (background processing)
-  CPU-bound on single core (no benefit)
-  Sequential tasks (can't parallelize)
-  High lock contention (overhead dominates)

Final Thought

Multithreading is a powerful tool, but not a magic solution. Understanding when and how to use it - and more importantly, when NOT to use it - is key to writing efficient, performant software.

The fundamental question: **Is my program waiting (I/O) or working (CPU)?**

- **Waiting?** Multithreading helps keep resources busy.
- **Working?** Multithreading only helps if you have multiple cores and the work can be divided.

Last updated: February 12, 2026